Activity 3 : Open/Closed Principle

and Liskov Substitution Principle

Leen Batta 12028719

violations of OCP:

the EmployeeManager class directly depends on the Manager class, so that when we need to add a new type of employee, we are required to modify the EmployeeManager class every time. and this violates the open closed principle.

violations of LSP:

there is a printEmployee method in EmployeeManager class and in employee class also but in the subclass Manager it lacks this method
So in the EmployeeManager class when it iterates through an array of employee objects when the object is Employee it prints the info by calling the method printEmployee but when the object is Manager , it has no such method

Refactoring:

create Printable interface
it includes a common method printDetails() can be implemented by various types of employees (ocp)

```java
interface Printable {
    void printDetails();
}
// Base class implementing the Printable interface
class Employee implements Printable {
    private String name;
    private String id;
    private double salary;

    public Employee(String name, String id, double salary) {
        this.name = name;
        this.id = id;
        this.salary = salary;
    }

    public String getName() {
        return name;
    }

    public String getId() {
        return id;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    @Override
    public void printDetails() {
        System.out.println("Name: " + name + ", ID: " + id + ", Salary: " + salary);
    }
}

```

The Manager class no longer extends the Employee class, to eliminate the LSP violation. It implements the interface and overrides the printDetails() method.

```
//Manager class implementing the Printable interface
class Manager implements Printable {
 private String name;
 private String id;
 private double salary;
 private String department;

 public Manager(String name, String id, double salary, String department) {
     this.name = name;
     this.id = id;
     this.salary = salary;
     this.department = department;
 }

 public String getDepartment() {
     return department;
 }

 public void setDepartment(String department) {
     this.department = department;
 }

 @Override
 public void printDetails() {
     System.out.println("Name: " + name + ", ID: " + id + ", Salary: " + salary + ", Department: " + department);
 }
}
```

The EmployeeManager class is now working with the Printable interface

```
//EmployeeManager class modified to work with Printable interface
class EmployeeManager {
 private Printable[] employees;
 private int currentIndex;

 public EmployeeManager(int size) {
     employees = new Printable[size];
     currentIndex = 0;
 }

 public void addEmployee(Printable printable) {
     employees[currentIndex++] = printable;
 }

 public void printDetails() {
     for (Printable printable : employees) {
         if (printable != null) {
             printable.printDetails();
         }
     }
 }
}
```

The updated main class after refactoring:

```
7  public class Main {
8      public static void main(String[] args) {
9          EmployeeManager em = new EmployeeManager(10);
0
1          Employee e1 = new Employee("John", "123", 50000);
2          Employee e2 = new Employee("Jane", "124", 55000);
3          Manager m1 = new Manager("Mike", "125", 70000, "Sales");
4          Manager m2 = new Manager("Mary", "126", 80000, "Marketing");
5
6          em.addEmployee(e1);
7          em.addEmployee(e2);
8          em.addEmployee(m1);
9          em.addEmployee(m2);
0
1          em.printDetails();
2      }
3  }
```